Goldsmiths
UNIVERSITY OF LONDON

Generative AI Preliminary Report

CM3070 Final Project Week 10

Author: Loke Shi Han, Sam
CM3070 Final Project
Module Leader: Dr Marco Gillies
Word count: 5669

1. Introduction

1.1. Project Template 4.1 - Orchestrating AI models to achieve a goal

This project follows the CM3020 Artificial Intelligence Template 4.1, orchestrating AI models to achieve a goal. The premise of the project is to combine pre-trained AI models to complete a goal. In this project, the AI model orchestration aims to ingest latest news articles and autonomously generate a podcast audio powered by a Recommender System, LLM model and Text to Speech model. This proposed system ultimately allows users to consume podcast audios generated from credible news articles that are personalised based on user preferences.

1.2. Overview

The aim for this preliminary report is to lay the groundwork and foundation for the AI-generated podcast project which takes in news articles and generates a two host podcast audio for multitaskers to listen to. Following the Discover, Design, Development, Deliver methodology introduced in the module, this report covers the discover and design phases. Specifically, this report focuses on literature review, project planning and a feature prototype.

The literature review supports the project by examining prior works that are related to the project. The literature review identifies related past projects, various techniques, evaluation metrics that are ideal for the system and ethical considerations of creating a system that generates news media using AI. The review also highlights potential challenges the system could face during implementation such as hallucinations of LLM models. These findings will subsequently shape the project plan, development and evaluation of the final project.

The project plan builds upon the knowledge from the literature review, highlighting the plan for the implementation of the full system. The project plan will focus on design choices, justifications, technologies, tools used and how the project will be realistically implemented and evaluated within the timeline. The full plan for the project is mapped out visually on a Gantt chart as well as a trello board to ensure timely deliverables.

The feature prototype further assesses the feasibility of the project, implementing the most technically challenging parts of the system. The prototype features a skeleton implementation of the news article to podcast pipeline with dummy data. Specifically, the LLM dialogue generator to Multi Text to Speech models and lastly serving the output through a NodeJS Express web server.

1.3. Motivation

This project is motivated by the fast changing use cases of artificial Intelligence models in the content creation domain. From AI generated images for storytelling to AI generated storylines, these technologies are quickly changing the landscape of how users produce and consume media. Amongst the emerging forms of modern day media, podcasts are becoming

widely accepted as a source of information. However, the production of podcasts still requires significant time to produce, making the information untimely. This project aims to explore whether AI can be effectively used to take in real time information and convert news articles to an engaging audio podcast for users to consume.

1.4. Project Concept

To create AI generated podcasts, the project should firstly include a recommender system to recommend news articles to users based on their personalised preference. The contents of the recommended news articles are then ingested by a conditioned LLM model to produce a conversational dialogue. The dialogue is then used by two instances of Text to Speech models to create a back and forth conversation between two voices. Additionally, an AI image generation model is used to create a cover image based on the contents of the news article. Ultimately, the user will be able to sign up for an account, set their preferences and the system will autonomously generate news podcasts for consumption.

1.5. Scope and Limitations

The scope and limitations of the projects are acknowledged before implementation to ensure that the project is realistically implemented. The scopes are defined as follows:

- Development of a system that ingests news articles and outputs an audio podcast
- Orchestration of multiple pre-trained AI models accessed using OpenRouter API
- Focus on English language podcasts
- Web application used for users to interact with the system.
- User's interaction triggers podcast generation to avoid redundant generation
- User is able to play and control podcast audio using web application
- User is able to download the podcast audio mp3 file

The limitations of this project are also recognised to ensure that the design and development of the project is feasible. The limitations are defined as follows:

- User evaluation for LLM model and TTS model is small scale, which may not reveal statistical information that would otherwise be present in a larger scale evaluation.
- The local machine used is a medium range laptop that is unable to train large scale models.
- The time limitations of this project may not allow for other advanced additions to the system.
- LLM models and TTS models are accessed via OpenRouter API but may be unavailable in the future if publishers decide to take it offline.
- Quantitative metrics of image generation model such as CLIPScore may not accurately identify adherence to the user prompt

2. Literature Review

2.1. Introduction to Literature Review

The purpose of this literature review is to evaluate and justify the key components of an AI generated podcast system that transforms recommended news articles into an audio podcast. Literature includes prior works, methods, implementation techniques and relevant domain research to further understand the project domain. By reviewing domain specific academic literature, the foundation, knowledge and motivation behind this project is strengthened.

Additionally, this review addresses ethical implications of automated content creation, specifically the credibility and misuse of AI generated content. Critical evaluations of existing approaches and technologies also identify current limitations in a multi agent AI model and highlights potential areas for future research. Ultimately, these insights help to motivate the development of an ethical, feasible and meaningful AI generated podcast system.

2.2. Prior Works

2.2.1. Traditional Podcasts

Podcasts are audio based media that feature speech and conversations by hosts and covers a wide range of topics like news, politics and culture. Since becoming popularised in the 2000s, podcasts have become a mainstream source of information. Familiarity with podcasts in the United States increased from 22% of individuals aged 12 and above in 2006 to 83% in 2023 (The Infinite Dial, 2023). This significant growth highlights the podcast's expanding role in media consumption and demonstrates its growing popularity.

Watson (2025) proposed news podcasts as a solution to news fatigue which is a rising issue that news outlets face. News fatigue is a phenomenon that happens when users are overwhelmed by constant bad news and start avoiding articles of current events (Moorefield, 2022). To address this, podcasts allow listeners to consume news conveniently whilst multitasking, which can appeal to users that lack the motivation to watch the news elsewhere. Podcasts are also accessible as they can be downloaded onto a device and listened to in any environment. Podcasts are also created in real time and released daily, which is timely compared to books or documentaries that require years to complete (Prince, 2020).

Despite the popularity of podcasts and its strengths, traditional podcasts are still constrained by production processes that require equipment, scripts, recording, editing and distribution to listeners (Buzzsprout, 2025). These laborious processes limit scalability and personalised content delivery to users. With this project, these limitations are addressed by automating the podcast creation process using AI by leveraging reliable news article recommendation, conversation generation and text to speech to create timely and personalised audio content without the labor of human production

2.2.2. Google Notebook LM Audio overview

In 2024, NotebookLM introduced an Audio Overview feature which generates conversational audio similar to podcasts based on uploaded sources (Wang, 2024). The feature uses state of the art Text-To-Speech and Large Language Models to produce a dialogue between two AI hosts that discusses the input source in a natural conversation. This feature is widely praised for being effective. Vigeant (2024) described his experience in his blog post as being “blown away”, highlighting the effectiveness and state of the art technologies used in the Audio Overview podcast feature. Therefore, this project should aim to achieve similar high quality results with Google NotebookLM as a benchmark for a good LLM summariser and text to speech model.

Whilst demonstrating the potential of AI generated podcasts and summarisation, it is limited by its input source. The feature relies heavily on the sources input by the user, meaning information credibility and quality is dependent on the sources provided. This ethical consideration is highlighted by Weidinger et al. (2022), retrieval-based generation has a risk of misinformation. In the context of news consumption, credibility is an important ethical consideration.

Additionally, the feature does not provide an end to end pipeline for podcast production. Users must still curate their own sources to upload and generate the audio overview. There is also no integration with a recommender system where personalised content is generated based on user preference. Therefore, despite its advanced Text to Speech and conversation generation capabilities, NotebookLM functions more like a summarization and research tool rather than a fully automated podcast system.

This project seeks to address these gaps by integrating curated news recommendation, conversation generation and text to speech models into an end to end system. Ultimately providing recommended news autonomously through a podcast audio.

2.3. Methods

2.3.1. Data Gathering

To populate the knowledge base with credible news articles to be recommended to users, an API is used instead of purely web scraping. Using a service such as NewsAPI is beneficial for its rich meta data fields such as title, published date and url (News API, n.d.). Additionally, the data response is in JSON format which simplifies ingestion. The data is then filtered and pre-processed before ingestion to ensure credible, non-duplicated news articles. Articles are sorted based on publishing data recency, ensuring that the ingested news are recent and relevant.

2.3.2. Data Storage

Due to the unstructured nature of the data required, which may include text-based news article content, multiple tags and prompts, a flexible storage solution is required. Thus, An unstructured Database Management System such as MongoDB is preferred. MongoDB is schema-less and supports JSON structured documents, which allow for new fields to be

added without changes to the entire schema. To prove this, Malik et al. (2020) experimented with NoSQL and SQL databases and found that the fastest one is MongoDB, making it a more ideal data storage solution for the data used in this project.

2.4. Techniques

2.4.1. News Recommender System

As described by Ricci et al. (2022), general personalised recommender systems are broadly categorized into either content based filtering and collaborative filtering, as well as a hybrid between the two methods.

Collaborative filtering generates recommendations based on patterns of ratings or usage. It relies heavily on high quality explicit user feedback, such as star or thumbs up ratings. It can also utilise implicit feedback such as user visit count that reflects user opinion. To perform well, a collaborative filtering system requires a large dataset of past user feedback and interaction which is not available. Making it unsuitable for this specific project as it would perform poorly due to the small user base.

Content based recommender systems generate personalised recommendations based on the similarity between the attributes of the user’s preferences and news articles. The result is a score that represents the target user’s level of interest in the article. However, content based filtering assumes that the user's preferences remain the same all the time (Ricci et al., 2022). This will be addressed by allowing the user to change their preferences whenever they would like, making it more dynamic to a user’s changing preferences.

2.4.2. LLM for Conversation Generation

To generate the podcast style conversation scripts, Large Language Models are accessed using OpenRoute API which provides access to multiple open source models. This enables experimentation of different free to use models without being constrained to a single LLM provider.

During experimentation, the models will be evaluated based on which one produced the most coherent, factual and natural sounding conversation. Zhu et al. (2024) experimented with using LLM to generate multi-turn dialogues, and found that proprietary models have better coherence and realism but open source models offer better customizability. Fine tuning would also significantly improve the performance of smaller language models.

Li & Gupta (2025) highlighted the gaps in the evaluation framework of conversation generation methodology and found poor correlation between automated metrics and human judgement of conversation quality. Additionally, See et al. (2019) found that human quality assessment depends on multiple dimensions that current automatic metrics are not able to comprehensively evaluate. Therefore, human evaluation is the most reliable method to assess conversation quality and realism. (Li & Gupta, 2025). The best model is selected based on this evaluation, ensuring a natural podcast like conversation.

2.4.3. Multi Text to Speech Model

The latest text to speech models include Tacotron 2, FastSpeech 2, VITS, and diffusion based models such as StyleTTS 2. Traditional models like tacotron use sequence to sequence architectures, whereas newer systems like VITS use variational inference and adversarial training to produce more natural sounding expressions. However newer systems can be more computationally expensive. Multiple TTS models will be evaluated to determine which one provides the best clarity, flow and speaker distinctiveness.

To evaluate the models, Mean Opinion Score is used, which is a widely used subjective metric to evaluate synthesized speech, usually regarded as the gold standard for evaluating text to speech models (Cooper & Yamagishi, 2023). MOS is ideal as it aligns with human perception and allows human evaluators to assess the clarity, tone and expressiveness more reliably than purely acoustic metrics like Mel Cepstral Distortion that rely on low level signal representations. Kongthaworn et al. (2021) highlights that modern TTS models have become very high quality and are indistinguishable by using only the low level signal representation. The TTS that achieves the highest MOS and most consistent voice quality will be used for the final system.

2.4.4. Image Generation

To generate podcast cover images, text to image diffusion models are used, such as Stable Diffusion and Dall-E 3. To identify the best model, CLIPScore is used. HuggingFace (n.d.) highlights that CLIPScore compares the semantic similarity between the prompt and the generated image. FID is not used to evaluate the images as it measures how similar 2 dataset of images are. It is widely used to compare between a set of real images and AI generated images, making FID favour realism over alignment to prompt (HuggingFace, n.d.). Therefore CLIPScore is more appropriate because the main requirement for the image generation model is that the image is semantically similar and relates to the news article. The model that achieves the highest CLIPScore, generates images adhering to the prompt is selected for the final system.

2.5. Ethics

Using generative AI to automate news based podcasts has several important ethical considerations. The primary concern is factual inconsistency. Large Language Models are known to produce confident but incorrect statements, especially when summarising or paraphrasing text content. In the context of news journalism, factual hallucinations can misinform users and undermine media integrity. (Olanipekun & Olakoyenikan, 2022). The Ministry of Digital Transformation of Ukraine (n.d.) emphasises that generative AI used in Ukrainian news media must have transparency and clarity by disclosing the use of AI and the methods of the use, which is a good practice to implement in this project. Additionally the source of the news article will also be included for users to fact check personally if needed.

Another ethical consideration relates to source reliability. Tools like NotebookLM allow users to upload their own sources, which may contain misinformation or biased perspectives. Without source validation, generative systems risk amplifying unreliable information, which is described by Ligteringen (n.d.) as “garbage in, garbage out”. In this project, this is addressed

by only retrieving articles published by trustworthy sources and providing source transparency to allow users to fact check if required.

Lastly, User privacy is an ethical consideration as personalisation is involved. While this project uses minimal preference setting, Chatterjee (2023) warns that a recommender system may have implications for user privacy. The data gathered could reveal sensitive behavioral patterns that may be harmful if leaked. Therefore, storage of user interaction and preferences should be done in a secure manner to address this ethical concern. In this project, measures will be taken to anonymise the data and ensure that bad actors are not able to misuse the data.

---

3. Project Plan

3.1. Project Design Introduction

This project design highlights the design and plan for the implementation of the Generative AI podcast project, focusing on design choices, justifications and how the project will be realistically implemented within the timeline. The project ultimately aims to autonomously generate multi host news podcasts from news articles using pre-trained AI models.

3.2. Domain and Users

The main domains for this project include the following:

- News media
- AI content creation
- Recommender Systems
- Text to Speech Technologies

The targeted users for this project include the following:

- Individuals who want autonomously generated news summarised into an audio podcast format
- News outlets or aggregators that want to add an audio podcast alongside their articles.

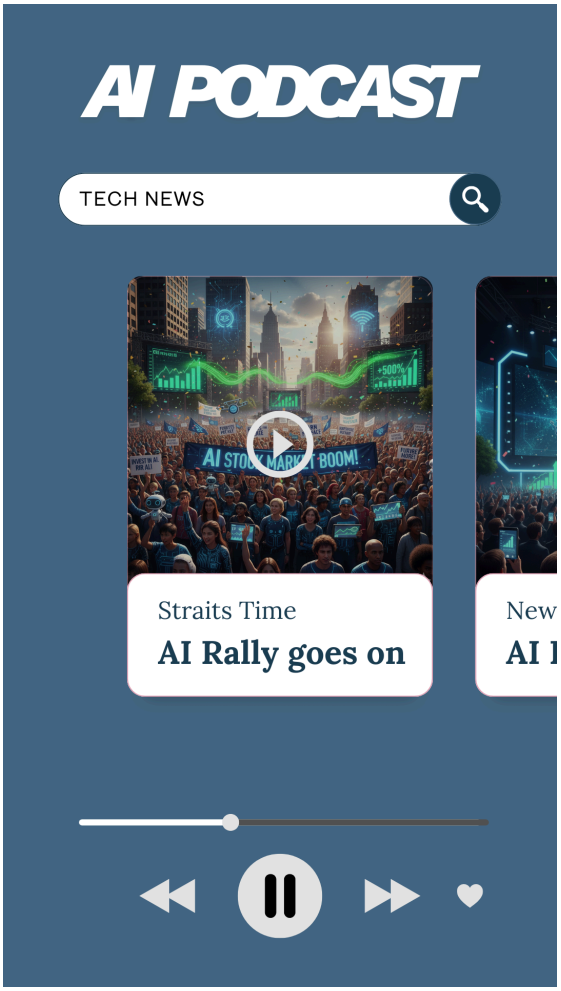
Each user has their own needs that must to be taken into consideration during implementation to ensure that they are met. For individuals, the system must be convenient to use without much user interaction to generate podcasts that match their preferences. The dialogue generated must also be accurate, with 2 different voices talking to each other for a logical conversation.

For Secondary users like news outlets and news aggregators that are featured by the system, the system should allow them to download the audio podcasts to feature alongside the original news article.

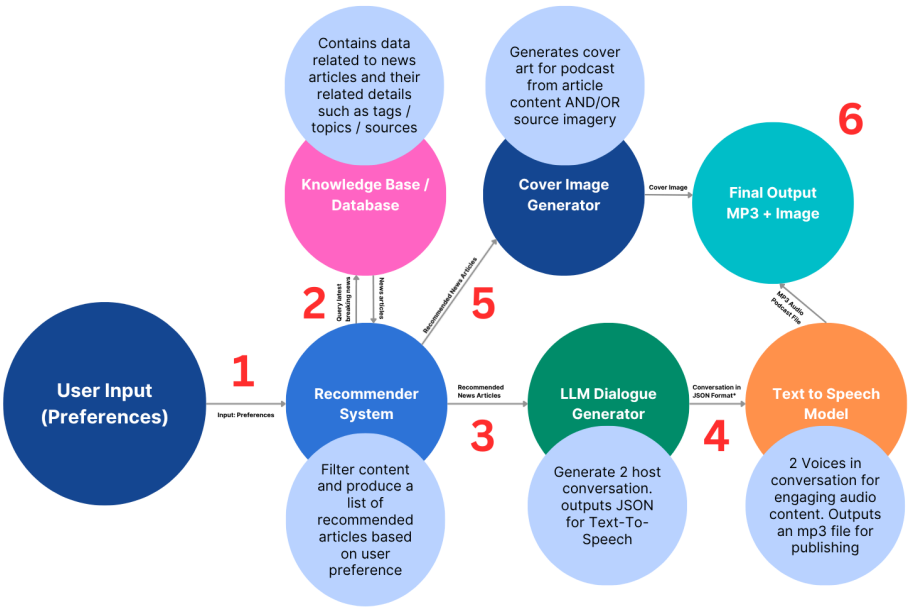
3.3. Architecture and Design Justification

3.3.1. User Interface

Motivated by the saying “don’t reinvent the wheel”, the User Interface is designed to be simple, modern and intuitive to ensure that users are able to quickly understand how to use the system. Additionally, having a simple user interface allows users to perform functions without much cognitive load, leading to more focus on their surroundings, making their commute safer.



3.3.2. Project Architecture



The full pipeline of the project is a straightforward flow that is efficient and keeps data safe by only exposing the user data to the recommender system. The full data flow and processes are numbered in the architecture diagram and functions accordingly:

1. The user's preferences are stored in a MongoDB database which is supported by the literature review to be a quicker way of retrieving unstructured data. User data includes preferences, past interactions and other related details for the recommender system.
2. The database also stores the latest news retrieved from NewsAPI which is retrieved daily. It is not done in real time to reduce loading time and API calls.
3. The recommender system will ingest the user's data and the latest news articles and output the recommended news articles in order of recommendation score.
4. The LLM model responsible for dialogue generation ingests the news article and generates a conversation in a JSON format that stores the conversation in chronological order. This JSON is then used by the 2 Text to Speech model to generate the conversation audio sentence by sentence.
5. The recommended news articles are also ingested by an image generator for a cover image.
6. The outputs are ultimately concatenated into a full conversation that is served to the user with a cover image and stored in the server for future retrieval if another user would like to listen to the same podcast.

3.4. Technologies & Methods

3.4.1. Hardware

The project is built using a consumer grade laptop that is capable of running python, database management software and a local web server. GPU training is not required as all

models are pre-trained or accessed via APIs. However, running the models locally on the device will also require significant processing power which can be handled with the laptop's medium range GPU.

3.4.2. Tech stack

For frontend, HTML, CSS and JS is used for a lightweight framework that is simple and capable of running audio players.

For backend, python is used for pipeline orchestration and the main language used to call the APIs and run the AI models. Python is used as most AI and Machine learning libraries are optimised for Python, making it easier to prototype, experiment and implement the project.

NodeJS with express is used for backend endpoints, communicating with MongoDB and running the local web server. NodeJS is used for its asynchronous nature, and ability to handle multiple concurrent user requests.

3.4.3. AI Models & Techniques

Recommender System: Content based filtering recommender system. News articles are given a score based on their embedding similarity with user preference. The similarity score is calculated using cosine similarity. The recommender system is evaluated using Precision at K which shows how many of the recommended news articles are relevant to the user's preferences.

LLM Dialogue Generator: Three latest open sourced LLM models will be accessed via OpenRouter. Supported by See et al. (2019), Human evaluation is best for model evaluation. The three models are evaluated based on multiple dimensions using a questionnaire and the best one selected, the questionnaire addresses the following:

- Coherence: Is it logical and consistent?
- Factuality: Does it hallucinate and spread misinformation?
- Natural Dialogue: Does the dialogue flow well?
- Latency: Does it generate dialogue quickly?

Text to Speech Model: Three latest open sourced TTS models are evaluated. As highlighted by Cooper & Yamagishi (2023), Mean Opinion Score is the gold standard for evaluating speech synthesis, where human evaluators assess the clarity, tone and expressiveness better than purely acoustic metrics. A questionnaire will be created for evaluators to address the following dimensions:

- Speech Naturalness: Does it sound natural?
- Voice stability: Is the tone, pace and style consistent?
- Latency: Does it generate audio quickly?

Image Generation: Three latest open sourced image generation models are evaluated. HuggingFace (n.d.) highlights that CLIPScore compares the semantic similarity between the prompt and the generated image which is the main requirement for this model in the

orchestration. The CLIPScore is calculated using a python library torchmetrics by PyTorch (Lightning AI, n.d.). The model with the highest CLIPScore is selected for the final system.

3.5. Evaluation

3.5.1. UI Evaluation

The UI is evaluated to ensure that it is accessible, easy to use and satisfactory to use. Because the target audience includes users who may be multitasking, the UI should minimise cognitive load and enable quick interactions to ensure safe multitasking. Therefore, the evaluation focuses on usability, accessibility and overall user satisfaction. This is done through a usability test, conducted with a small group of participants who match the target audience. Users will be asked to complete a set of tasks using the system and complete a questionnaire that addresses the following:

- Task completion rate: Able to complete the task?
- Completion time: How much time taken to complete a task?
- Number of errors: Errors when completing tasks?
- Learnability: Easy to complete task?
- Likelihood of repeated use: Satisfied with usage?

Additionally, Accessibility testing is done to ensure inclusivity. The basic accessibility criteria to meet according to WCAG (2025) is as follows:

- Text to background contrast ratio of 4.5:1
- Minimally 18 point font size
- Audio controls required
- Colour choices should not disadvantage colour blind users

These evaluation tests will take place multiple times during development to ensure an iterative approach to UI design. The feedback from each set of evaluation will identify gaps and areas of improvements.

3.5.2. Backend Evaluation

The backend systems are evaluated to ensure that data retrieval and API interactions are efficient, reliable and robust enough to support the full end to end pipeline. As the system heavily depends on external APIs and database queries, the evaluation will identify the advantages and shortcomings of the performance and reliability of the backend services.

For the data retrieval from MongoDB and NewsAPI, response times will be measured and logged for common queries. The average query times and success rate will be recorded to determine if the database and APIs perform well under real world usage.

Additionally, API endpoints supported by NodeJS and Express will be evaluated on response time. For each end point the average time from request to response is measured, particularly important end points such as end points that trigger the full pipeline.

3.5.3. AI Model Evaluation

AI Model evaluation will occur at two stages, during model selection and full system testing. Each model will be evaluated using the respective metrics mentioned in the AI model and technique section. The models will be evaluated before and after implementation. The testing during model selection determines which model is the best fit for the project. The full system testing evaluates each model's performance in a real world context with real news articles and user preferences.

5.4. Testing Strategy
To efficiently and effectively test the system, testing will occur at three levels, unit testing, integration testing and full system testing.

Unit testing will be used to test small isolated components, following the test driven development structure to ensure that every module works as it should before integrating the system. Additionally, when tests are written before code and just enough code is written to pass the tests, redundancy is decreased, making the code base more efficient.

Integration testing will evaluate how well the components work together, particularly between modules such as LLM Dialogue Generator with Text to Speech model. One such foreseeable example is the passing of the dialogue to the text to speech model not being in a consistent JSON format which causes the text to speech model to throw exceptions. Thus, Integration testing will be done using test articles and dummy user profiles to simulate real usage before deployment.

System testing will be done to evaluate the entire system, including but not limited to the UI, AI model performance and user satisfaction. This system test should ensure that the entire pipeline consistently completes without errors and that a valid podcast audio and cover image is produced and served to the user.

3.6. Project Plan

This project follows the Discover, Design, Development, Delivery methodology introduced in the module lecture.

3.6.1. Tools used

To ensure consistent progress and boost productivity, multiple tools introduced throughout the course are used.

- Trello for task tracking (Similar to Kanban board)
- Gantt Chart for timeline planning
- Google Docs for documentation and report writing
- Github for version control

3.6.2. Project plan

A list of tasks are compiled to ensure that the project adheres to the submission timeline. Each task should take no more than one week to provide small milestones, making each small achievement meaningful and motivating.

Task	Start	Finish
Discover Phase		
Explore project ideas	15 Oct 2025	15 Oct 2025
Identify 3 project templates	16 Oct 2025	16 Oct 2025
Choose final project template	17 Oct 2025	17 Oct 2025
Project Research	18 Oct 2025	24 Oct 2025
Submit proposal slides for review	24 Oct 2025	26 Oct 2025
Project proposal video	27 Oct 2025	27 Oct 2025
Ethics research	28 Oct 2025	30 Oct 2025
Complete literature review	31 Oct 2025	7 Nov 2025
Design Phase		
Project Plan Skeleton	8 Nov 2025	15 Nov 2025
Design diagrams, wireframes and charts	16 Nov 2025	24 Nov 2025
Complete project plan	25 Nov 2025	2 Dec 2025
Prototype skeleton	3 Dec 2025	7 Dec 2025
Prototype 1st iteration	8 Dec 2025	11 Dec 2025
Prototype evaluation	12 Dec 2025	12 Dec 2025
Prototype 2nd iteration (Optional)	13 Dec 2025	17 Dec 2025
Preliminary Report and Prototype video	17 Dec 2025	17 Dec 2025
Development Phase		
Set up Github Repository	18 Dec 2025	18 Dec 2025
AI Model evaluation and selection	19 Dec 2025	22 Dec 2025
Set up MongoDB Environment	23 Dec 2025	23 Dec 2025
Set up NodeJS Environment	23 Dec 2025	23 Dec 2025
Set up Python Environment	23 Dec 2025	23 Dec 2025
Set up Frontend skeleton	23 Dec 2025	23 Dec 2025
Backend - Python NewsAPI ingestion into MongoDB	24 Dec 2025	26 Dec 2025
Frontend - User Sign up and Login	27 Dec 2025	29 Dec 2025
Fullstack - User Preference setting upload	30 Dec 2025	2 Jan 2026
Backend - Python Recommender System	3 Jan 2026	8 Jan 2026
Backend - Python LLM Dialogue Generator	9 Jan 2026	13 Jan 2026
Backend - Python Multi TTS model	14 Jan 2026	18 Jan 2026
Backend - Python Image generation	19 Jan 2026	23 Jan 2026
Backend - Pipeline integration (Recommender to LLM to TTS and Image Generator to server storage)	24 Jan 2026	28 Jan 2026
Middleware - Trigger pipeline and generate podcasts	29 Jan 2026	1 Feb 2026
Frontend - podcast generation function	2 Feb 2026	5 Feb 2026
Frontend - Playback function and user feedback of recommendation	6 Feb 2026	9 Feb 2026
System evaluation	10 Feb 2026	17 Feb 2026
User evaluation	18 Feb 2026	25 Feb 2026
Buffer for second development iteration	25 Feb 2026	5 Mar 2026
Buffer for second evaluation iteration	6 Feb 2026	15 Mar 2026
Delivery Phase		
Examination	16 Mar 2026	16 Mar 2026
Draft Report	15 Jan 2026	9 Feb 2026
Final Report and submission	10 Feb 2026	23 Mar 2026

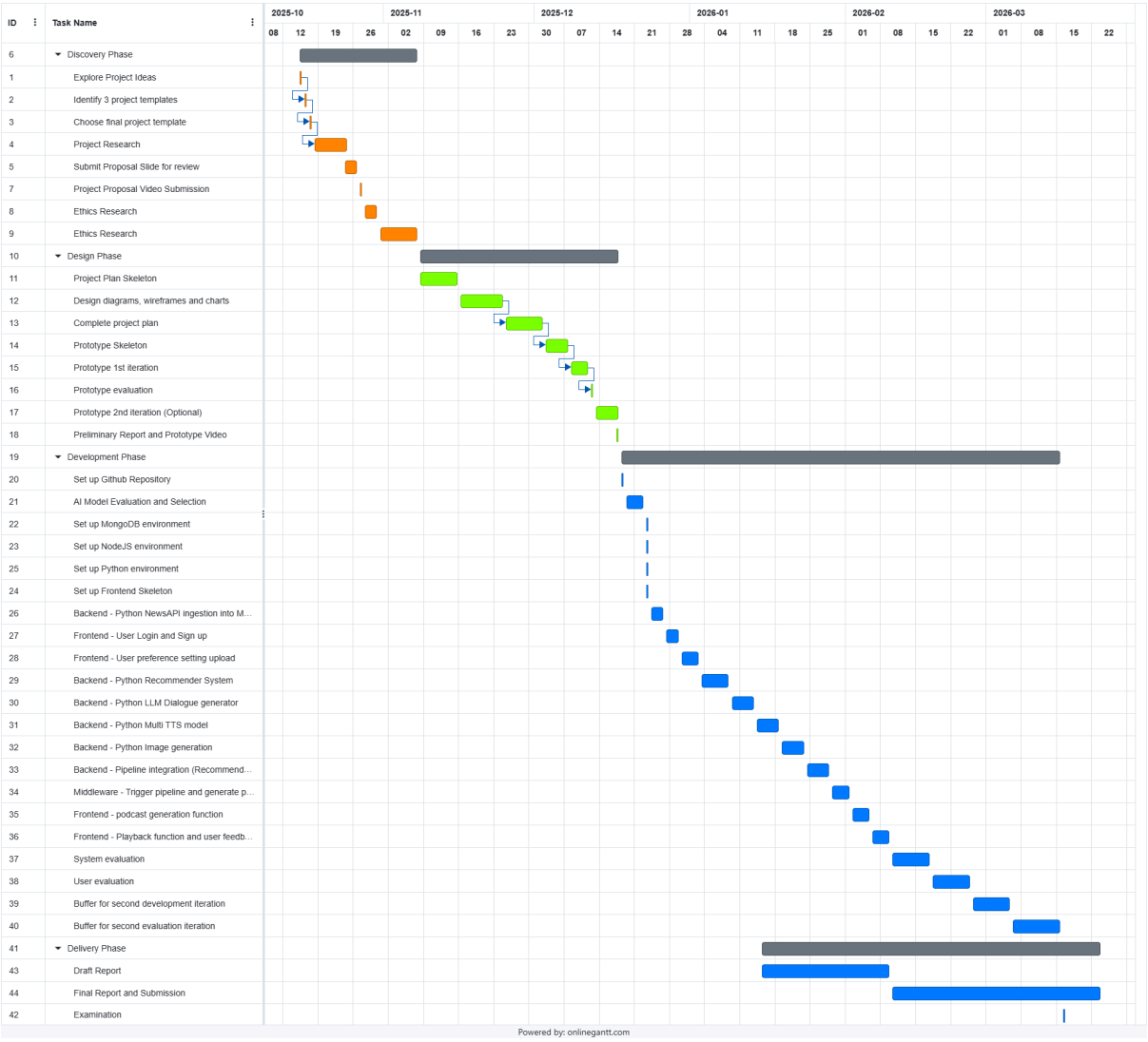
3.6.4. Contingency Plans

In the case of delays or inability to complete the project due to time limitations, core functions are prioritised to demonstrate the main concept of the system. Which is to generate audio podcasts from the news articles. Cover image generation is therefore optional and could be excluded from the final demonstration. A second iteration of the development and evaluation cycle may also be foregone if time becomes limited.

Fallbacks models for LLM, TTS and Image generation will also be chosen during model selection phase, in case the publishers of the model take the model off OpenRouter API. A simple fallback recommender system may also be needed if the recommender is not completed, which will use rule based filtering by keyword, ensuring the user still receives relevant content without an advanced recommender system.

3.6.3. Gantt Chart

A Gantt Chart is created to visualise the expected timeline and promotes accountability. Additionally, there are added buffer periods in case of unexpected delays or difficulties.



3.6.4. Github

Github is used for version control to manage the source code during development. The repository created will contain all code and documents relevant to the project. As the development only features 1 developer, there will be no branching to simplify the repository. However, descriptive comments will be committed frequently to explain what was implemented or fixed, making it easier to revert to earlier versions if something happens. Github will also serve as an additional backup, in case something were to happen to the local machine.

3.6.5. Success Criteria

To consider the project successful, the following criterias are defined:

- Podcast generated in less than 5 minutes
- 80% Score for LLM human evaluation
- 80% Mean Opinion Score for TTS model
- 80% UI testing score
- Pipeline of AI models is functional and gracefully handles errors

4. Feature Prototype

4.1. Feature Prototype Introduction

To assess whether the project pipeline is feasible, a feature prototype is created, highlighting the most technically challenging parts that are the hardest to implement. This prototype includes a skeleton implementation of the news article to podcast generation, specifically the LLM dialogue generator to Multi text to speech models and lastly serving the output on a web server.

4.2. Dummy data

To test the prototype, dummy data is collected from NewsAPI's documentation to ensure that the prototype runs on data accurate to NewsAPI's response. This data is stored in LLM.py for easy access if the prompt needs to be adjusted.

4.3. LLM Model

To access free open source LLM Models, an API call is made to OpenRouter API. Using pre-trained open source models allows a light weight and quick implementation without large amounts of resources being used for training. For the prototype, I used Amazon's latest free reasoning LLM model "Nova 2 Lite". The LLM model is first individually tested to ensure that it works before integrating into a pipeline. The output of the model should be a chronological list of host dialogue, for the text to speech script to loop through and alternate between voices.

```python
def generate_podcast_script(article_content):

    openrouter_payload = {
        "model": "amazon/nova-2-lite-v1:free",
        "messages": [
            {
                "role": "system",
                "content": "You are a podcast script generator. Your task is to create a conversational podcast
            },
            {
                "role": "user",
                "content": str(content_json)
            }
        ],
        # set max token to 0 to ensure no limit on response length
        "max_tokens": 0
}

    response = requests.post(API_URL, headers=headers, json=openrouter_payload)

    if response.status_code != 200:
        print("Request failed:")
        print(response.status_code, response.text)
    else:
        data = response.json()
        print("raw response data:")
        print(data)
        print("====================")
        reply = data["choices"][0]["message"]["content"]
        print("LLM response:")
        print(reply)

if __name__ == "__main__":
    generate_podcast_script(content_json)
```

To ensure that the output is consistent and deterministic so that the Text to Speech script is able to access and loop the output, the model is instructed in the system prompt sent in the API payload to adhere to a specified format.

```
You must return your answer strictly in a list and nothing else. The list must follow this exact structure: ['This is the first sentence','this is the
```

The model successfully generated a list populated with alternating dialogue.

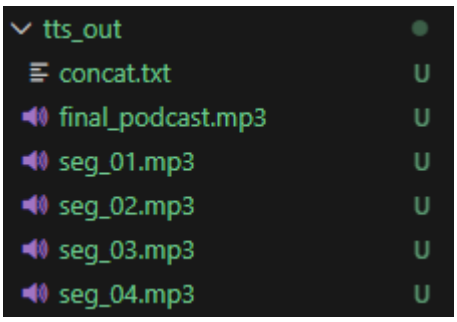
```
====================
LLM response:
['Welcome back to another edition of Market Watch, I\'m your host Alex.', 'And I\'m Sam. Today we\'re diving into some exciting news from Broadcom that could have major implications for Nvidia
nvestors.', 'That\'s right, Sam. Broadcom\'s CEO, Hock Tan, just shared some incredible insights that are sending shockwaves through the tech sector.', 'He essentially confirmed what many analy
ts have suspected: AI adoption is accelerating faster than anyone expected, especially in data centers and semiconductor spaces.', 'Exactly. This is huge because it directly benefits Nvidia, wh
se GPUs are the backbone of AI computing. The evidence Hock Tan provided shows we\'re entering a new era of AI integration.', 'Right, and for investors, this means Nvidia\'s dominance in the AI
 'cost_details': {'upstream_inference_cost': None, 'upstream_inference_prompt_cost': 0, 'upstream_inference_completions_cost': 0}, 'completion_tokens_details': {'reasoning_tokens': 0, 'image_to
ens': 0}}}
====================
```

4.4. Text to Speech Model

A python library edge-tts by Microsoft is used to implement the prototype text to speech model. This model is used for its convenience and easy implementation in python. The script loops through the dialogue list generated by the LLM model, alternating between two voices and generating audio segments for each line. Finally, ffmpeg is used to stitch audio snippets together to create a full podcast output from the audio segments.

```python
dialogue = [
    "Hello this is the first speaker for the prototype testing",
    "And this is the second speaker responding to the first",
    "Now the first speaker has something more to add",
    "Finally the second speaker concludes the prototype test"
]

voices = [
    "en-US-JennyNeural",
    "en-US-GuyNeural"
]

output_directory = Path("tts_out")
final_output = output_directory / "final_podcast.mp3"

async def speak_one_line(text, voice, out_path):
    tts = edge_tts.Communicate(text=text, voice=voice)
    await tts.save(str(out_path))

async def main():
    output_directory.mkdir(exist_ok=True)

    all_speech_files = []

    for i, line in enumerate(dialogue, 1):
        output = output_directory / f"seg_{i:02d}.mp3"
        await speak_one_line(line, voices[i % len(voices)], output)
        all_speech_files.append(output)

    # Create concatenated list for ffmpeg to stitch together
    # Cannot use all_speech_files directly because ffmpeg takes in a txt file
    concat_file = output_directory / "concat.txt"
    with open(concat_file, "w", encoding="utf-8") as f:
        for seg in all_speech_files:
            f.write(f"file '{seg.resolve()}'\n")

    # Stitch using ffmpeg
    subprocess.run([
        "ffmpeg",
        "-y",
        "-f", "concat",
        "-safe", "0",
        "-i", str(concat_file),
        "-c", "copy",
        str(final_output)
    ], check=True)
```

The TTS model successfully generate segment audios and a concatenated podcast audio file “final_podcast.mp3”



4.5. Integration of LLM Model with Text to Speech Mode

The modular LLM dialogue generation process and Text to Speech process is then modularized into a reusable module for import and called in a pipeline file “pipeline.py” that will be triggered by the web server upon user request. The separation of concern in this prototype is especially useful in future work later in the project as specific modules could be reused and improved without the need to change other parts of the system.

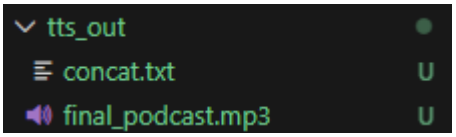
```python
# function to generate audio by looping through dialogue and stitching segments
async def run_tts(dialogue, output_directory):
    output_directory.mkdir(exist_ok=True)
    final_output = output_directory / "final_podcast.mp3"

    all_speech_files = []

    for i, line in enumerate(dialogue):
        output = output_directory / f"seg_{i+1:02d}.mp3"
        await speak_one_line(line, voices[i % len(voices)], output)
        all_speech_files.append(output)

    # Create concatenated list for ffmpeg to stitch together
    # Cannot use all_speech_files directly because ffmpeg takes in a txt file
    concat_file = output_directory / "concat.txt"
    with open(concat_file, "w", encoding="utf-8") as f:
        for seg in all_speech_files:
            f.write(f"file '{seg.resolve()}'\n")

    # Stitch using ffmpeg
    subprocess.run([
        "ffmpeg",
        "-y",
        "-f", "concat",
        "-safe", "0",
        "-i", str(concat_file),
        "-c", "copy",
        str(final_output)
    ], check=True)

    return final_output

# main function to be called in pipeline.py
def generate_podcast_audio(dialogue, output_directory='tts_out'):
    output_directory = Path(output_directory)
    final_output_path = asyncio.run(run_tts(dialogue, output_directory))
    return str(final_output_path.resolve())
```

```python
# Function to parse the LLM response into a dialogue list
def parse_llm_dialogue(reply):
    reply = reply.strip()

    #try to parse using ast.literal_eval
    try:
        data = ast.literal_eval(reply)
        if isinstance(data, list) and all(isinstance(item, str) for item in data):
            return data
    except Exception as e:
        print("Error when parsing LLM response:", e)

def generate_podcast_script(article_content):

    openrouter_payload = {
        "model": "amazon/nova-2-lite-v1:free",
        "messages": [
            {
                "role": "system",
                "content": "You are a podcast script generator. Your task is to create a
            },
            {
                "role": "user",
                "content": str(content_json)
            }
        ],
        # set max token to 0 to ensure no limit on response length
        "max_tokens": 0
}

    response = requests.post(API_URL, headers=headers, json=openrouter_payload)

    if response.status_code != 200:
        print("Request failed:")
        print(response.status_code, response.text)
    else:
        data = response.json()
        print("====================")
        reply = data["choices"][0]["message"]["content"]
        print("LLM response:")
        print(reply)
        return parse_llm_dialogue(reply)
```

The pipeline.py script then imports these modules and orchestrates them into a pipeline

```python
def main():
    dummy_news_article = content_json

    #generate dialogue using LLM.py
    dialogue = generate_podcast_script(dummy_news_article)
    podcast_audio_path  = generate_podcast_audio(dialogue, output_directory='tts_out')

    results = {
        "response_code": 200,
        "title": dummy_news_article['title'],
        "dialogue": dialogue,
        "podcast_audio_path": podcast_audio_path
    }

    print(results)

if __name__ == "__main__":
    main()
```

The pipeline integration script was executed successfully with a full concatenated podcast audio file generated in the tts_out folder.



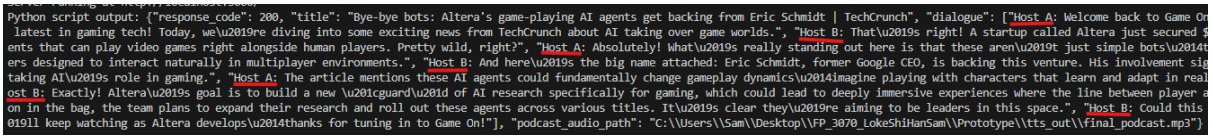
4.6. Connecting pipeline with Frontend

To confirm that the user is able to trigger the pipeline through the browser, a simple front end UI is created. The UI features a simple button that will send a GET /generate request to the NodeJS server. The endpoint route triggers the podcast generation process which responds with the path directory of the generated mp3 file. Which can then be played and controlled using HTML inbuilt audio playback control.

Download is also available through the /download/:filename end point which sends a download response to the client. This is useful for users who would like to listen to podcasts offline, making it more accessible in environments with no internet connectivity.

4.7. Prototype Evaluation

Despite only featuring the skeleton of the project, the prototype demonstrates the feasibility of the project as well as some shortcomings to be addressed in the final submission. Firstly, it is observed that seldom the LLM model will include a speaker descriptor in the conversation, such as “Host A” or “Host 1”. This causes the Text to Speech model to read out the host descriptor each time the text to speech model alternates speakers, making the podcast hard to follow and contain unmeaningful words. To address this, a more strict and detailed system prompt must be used to ensure the dialogue generated adheres to the specified format.


Python script output: {"response_code": 200, "title": "Bye-bye bots: Altera's game-playing AI agents get backing from Eric Schmidt | TechCrunch", "dialogue": ["Host A: Welcome back to Game On
latest in gaming tech! Today, we\u2019re diving into some exciting news from TechCrunch about AI taking over game worlds.", "Host B: That\u2019s right! A startup called Altera just secured $
ents that can play video games right alongside human players. Pretty wild, right?", "Host A: Absolutely! What\u2019s really standing out here is that these aren\u2019t just simple bots\u2014t
ers designed to interact naturally in multiplayer environments.", "Host B: And here\u2019s the big name attached: Eric Schmidt, former Google CEO, is backing this venture. His involvement sig
taking AI\u2019s role in gaming.", "Host A: The article mentions these AI agents could fundamentally change gameplay dynamics\u2014imagine playing with characters that learn and adapt in real
ost B: Exactly! Altera\u2019s goal is to build a new \u201cguard\u201d of AI research specifically for gaming, which could lead to deeply immersive experiences where the line between player a
on in the bag, the team plans to expand their research and roll out these agents across various titles. It\u2019s clear they\u2019re aiming to be leaders in this space.", "Host B: Could this
019ll keep watching as Altera develops\u2014thanks for tuning in to Game On!"], "podcast_audio_path": "C:\\Users\\Sam\\Desktop\\FP_3070_LokeShiHanSam\\Prototype\\tts_out\\final_podcast.mp3"}

Additionally, the loading time for the podcast to be generated is lengthy, making the user experience unsatisfactory. This is mainly caused by the LLM model because the full response must be completed before sending the conversation to the Text to Speech model. To potentially address this in the full project, the final podcast audio file generated will be saved in the server, for future users to retrieve quickly. Making the long wait occur only once per unique article.

The Text to Speech model works well, but sounds robotic and unnatural. This is addressed during model selection in the next phase, where different Text to Speech models are evaluated and the best selected for final implementation.

4.8. Prototype Conclusion

Through the literature review, project design and feature prototype, a strong foundation is laid out for the completion of the project. The literature review successfully supported and justified the project’s domain, user, techniques and ethical considerations. A project plan is then created to ensure that the tools, methods and techniques are justified and achievable. Lastly, the feature prototype demonstrated the feasibility of the project by successfully implementing the LLM, TTS and Frontend pipeline, which is the most technically challenging portion of the project system architecture.

5. References

1. The Infinite Dial. (2023). The Podcast Consumer 2023. https://www.edisonresearch.com/wp-content/uploads/2023/03/The-Podcast-Consumer-2023-1.pdf
2. Watson, A. (2025, April 25). *Topic: News and podcasts*. Statista. https://www.statista.com/topics/12838/news-and-podcasts/#topicOverview
3. Prince, B. F. (2020). Podcasts: The potential and possibilities. *Teaching Sociology*, *48*(4), 269–271. https://doi.org/10.1177/0092055x20959837
4. Moorefield, N. (2022, September 16). News fatigue: What it is and how to avoid it | UNC-chapel hill. https://www.unc.edu/discover/news-fatigue-what-it-is-and-how-to-avoid-it/
5. Buzzsprout. (2025, January 1). *How to start a podcast: Complete Step-by-step guide [2025]*. https://www.buzzsprout.com/blog/how-to-start-a-podcast
6. Wang, B. (2024, September 11). *Notebooklm now lets you listen to a conversation about your sources*. Google. https://blog.google/technology/ai/notebooklm-audio-overviews/
7. Vigeant, E. (2024, October 17). *Testing Notebooklm's audio overview feature*. Okiok. https://www.okiok.com/testing-notebooklms-audio-overview-feature/
8. Weidinger, L., Mellor, J., Rauh, M., Griffin, C., Uesato, J., Huang, P.-S., Cheng, M., Glaese, M., Balle, B., Kasirzadeh, A., Kenton, Z., Brown, S., Hawkins, W., Stepleton, T., Biles, C., Birhane, A., Haas, J., Rimell, L., Hendricks, L. A., Isaac, W., Legassick, S., Irving, G., & Gabriel, I. (2021, December 8). *Ethical and social risks of harm from language models*. arXiv. https://arxiv.org/abs/2112.04359
9. *News API – search news and blog articles on the web*. News API Search News and Blog Articles on the Web. (n.d.). https://newsapi.org/
10. Malik, A., Burney, A., & Ahmed, F. (2020). A comparative study of unstructured data with SQL and NO-SQL database management systems. *Journal of Computer and Communications*, *08*(04), 59–71. https://doi.org/10.4236/jcc.2020.84005
11. Ricci, F., Rokach, L., & Shapira, B. (2022). *Recommender Systems Handbook*. Springer US Springer.
12. Li, S., & Gupta, A. (2025, August 4). *Can LLMs Generate High-Quality Task-Specific Conversations?*. Can llms generate high-quality task-specific conversations? https://arxiv.org/html/2508.02931v1#bib.bib52
13. See, A., Roller, S., Kiela, D., & Weston, J. (2019, June). What makes a good conversation? how controllable attributes affect human judgments. *Proceedings of the 2019 Conference of the North*, 1702–1723. https://doi.org/10.18653/v1/n19-1170
14. Cooper, E., & Yamagishi, J. (2023, August 24). Investigating range-equalizing bias in mean opinion score ratings of synthesized speech. *INTERSPEECH 2023*, 1104–1108. https://doi.org/10.21437/interspeech.2023-1076
15. Kongthaworn, T., Naowarat, B., & Chuangsuwanich, E. (2021). Spectral and Latent Representation Distortion for TTS evaluation. *Spectral and Latent Speech Representation Distortion for TTS Evaluation*. https://doi.org/10.58837/chula.the.2021.101
16. Hugging Face. (n.d.). *Evaluating diffusion models*. Evaluating Diffusion Models. https://huggingface.co/docs/diffusers/main/en/conceptual/evaluation

17. Olanipekun, S. O., & Olakoyenikan, O. (2022, December 26). Ethical implications of generative AI in journalism: Balancing innovation, truth, and Public Communication Trust. *World Journal of Advanced Research and Reviews*, *16*(3), 1293–1311. https://doi.org/10.30574/wjarr.2022.16.3.1159
18. Ministry of Digital Transformation of Ukraine. (n.d.). Guidelines on the responsible use of artificial intelligence in the news media. https://rm.coe.int/guidelines-on-the-responsible-use-of-artificial-intelligence-in-the-ne/1680b09029
19. Ligteringen, J. (n.d.). *Everything wrong with retrieval-augmented generation*. Leximancer. https://www.leximancer.com/blog/everything-wrong-with-retrieval-augmented-generation
20. Chatterjee, S. (2023, July 7). *Recommendation systems: Ethical challenges and the regulatory landscape*. Holistic AI. https://www.holisticai.com/blog/recommendation-systems
21. Lightning AI. (n.d.). CLIP score. CLIP Score - PyTorch-Metrics 1.8.2 documentation. https://lightning.ai/docs/torchmetrics/stable/multimodal/clip_score.html
22. World Wide Web Consortium. (2022, May 6). Web content accessibility guidelines (WCAG) 2.1. W3C. https://www.w3.org/TR/WCAG21/